

Spring Security

PART 1: INTRODUCTION TO SPRING SECURITY

◆ What is Spring Security?

Spring Security is a **powerful and customizable authentication and access-control framework** for Java applications, especially Spring-based apps.

It provides:

- **Authentication** → Verifying *who* the user is.
- **Authorization** → Determining *what* the user can access.

It's part of the **Spring Framework ecosystem** and integrates seamlessly with **Spring Boot**, **Spring MVC**, and even **reactive applications (WebFlux)**.

⚙️ PART 2: WHY SPRING SECURITY?

Problem	Without Spring Security	With Spring Security
Authentication	Manual verification using filters, servlets, DB queries	Ready-made mechanisms (form, basic, JWT, OAuth2, LDAP)
Authorization	Manually manage access	Annotations and configurations handle it easily
Passwords	Plain text or custom hashing	BCrypt hashing out of the box
CSRF, CORS, Session Fixation	Must handle manually	Automatic protection
Integration with OAuth2, JWT	Tedious	Prebuilt modules

✅ In short — it's a **security umbrella** covering every layer of your app.

🧩 PART 3: CORE CONCEPTS

Let's break Spring Security into **fundamental building blocks**.

1 Security Filter Chain

- Core of Spring Security.
- Every request passes through a **chain of filters** before reaching the controller.

Example (Simplified chain):

```
UsernamePasswordAuthenticationFilter →  
BasicAuthenticationFilter →  
BearerTokenAuthenticationFilter (for JWT) →  
SecurityContextPersistenceFilter →  
FilterSecurityInterceptor →  
DispatcherServlet
```

 The filter chain is configured automatically when you include `spring-boot-starter-security`.

2 SecurityContext & SecurityContextHolder

- `SecurityContext` holds the **Authentication** object (details about the logged-in user).
- Stored in `ThreadLocal` using `SecurityContextHolder`.

Example:

```
Authentication auth = SecurityContextHolder.getContext().getAuthentication  
();  
String username = auth.getName();
```

✅ **Key point:** Once authenticated, Spring stores user info in this context for the entire request lifecycle.

3 Authentication

Represents the identity of the user.

Interface:

```
public interface Authentication extends Principal, Serializable {  
    Collection<? extends GrantedAuthority> getAuthorities();  
    Object getCredentials();  
    Object getDetails();  
    Object getPrincipal();  
    boolean isAuthenticated();  
}
```

Implementation examples:

- `UsernamePasswordAuthenticationToken`
- `JwtAuthenticationToken`
- `OAuth2AuthenticationToken`

4 AuthenticationManager

Responsible for authenticating a user.

```
public interface AuthenticationManager {  
    Authentication authenticate(Authentication authentication)  
        throws AuthenticationException;  
}
```

- Delegates to `AuthenticationProvider`.
- Returns a **fully authenticated** object if successful.

5 AuthenticationProvider

Performs the **actual verification** (e.g., check DB credentials).

Example:

```

public class MyAuthProvider implements AuthenticationProvider {
    @Override
    public Authentication authenticate(Authentication auth) {
        String username = auth.getName();
        String password = auth.getCredentials().toString();
        // Verify username/password
    }
}

```

Multiple providers can exist (for DB, LDAP, JWT, etc.).

6 UserDetails & UserDetailsService

Spring uses these to fetch user information.

```

public interface UserDetails {
    String getUsername();
    String getPassword();
    Collection<? extends GrantedAuthority> getAuthorities();
}

```

`UserDetailsService` is the bridge between **your DB** and Spring Security.

Example:

```

@Service
public class MyUserDetailsService implements UserDetailsService {
    @Override
    public UserDetails loadUserByUsername(String username) {
        return new User("admin", passwordEncoder.encode("1234"),
            List.of(new SimpleGrantedAuthority("ROLE_ADMIN")));
    }
}

```

```
}
```

7 PasswordEncoder

For password hashing.

Commonly used: `BCryptPasswordEncoder`.

```
@Bean
public PasswordEncoder passwordEncoder() {
    return new BCryptPasswordEncoder();
}
```

✓ Never store plain passwords.

8 GrantedAuthority

Represents permissions or roles assigned to the user.

- Example: `ROLE_USER`, `ROLE_ADMIN`.

PART 4: AUTHENTICATION FLOWS

◆ 1. Form Login

Browser-based login (HTML form).

Config:

```
@Bean
SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
    return http
        .authorizeHttpRequests(auth → auth
            .requestMatchers("/admin/**").hasRole("ADMIN")
            .anyRequest().authenticated())
        .formLogin(withDefaults())
}
```

```
.build();  
}
```

Flow:

1. User submits login form.
2. `UsernamePasswordAuthenticationFilter` triggers.
3. `AuthenticationManager` validates credentials.
4. `SecurityContextHolder` stores authenticated user.
5. Redirects to secured page.

◆ 2. Basic Authentication

For APIs (credentials sent in `Authorization: Basic base64(user:pass)`).

```
http.httpBasic(Customizer.withDefaults());
```

◆ 3. JWT (JSON Web Token)

Stateless authentication for REST APIs.

Flow:

1. User logs in → JWT created and returned.
2. Client sends JWT in header (`Authorization: Bearer <token>`).
3. Filter validates JWT → sets Authentication in `SecurityContextHolder` .

Custom Filter Example:

```
public class JwtAuthFilter extends OncePerRequestFilter {  
    @Override  
    protected void doFilterInternal(HttpServletRequest req, HttpServletResponse res, FilterChain chain)  
        throws ServletException, IOException {
```

```
String token = getToken(req);
if (validate(token)) {
    UsernamePasswordAuthenticationToken auth = ...
    SecurityContextHolder.getContext().setAuthentication(auth);
}
chain.doFilter(req, res);
}
}
```

◆ 4. OAuth2 / OpenID Connect

Used for social login (Google, GitHub, Facebook).

Config:

```
http
    .oauth2Login(oauth → oauth.loginPage("/login"));
```

Spring Security automatically handles token exchange, redirection, and user details mapping.

◆ 5. LDAP Authentication

Integrates with enterprise directory services.

Config:

```
http
    .ldapAuthentication(ldap → ldap
        .userDnPatterns("uid={0},ou=people")
        .groupSearchBase("ou=groups")
        .contextSource()
        .url("ldap://localhost:8389/dc=springframework,dc=org"));
```

PART 5: AUTHORIZATION (ACCESS CONTROL)

◆ Using `@PreAuthorize` and `@PostAuthorize`

```
@PreAuthorize("hasRole('ADMIN')")
@GetMapping("/users")
public List<User> getAllUsers() { ... }
```

Enable in config:

```
@EnableMethodSecurity
```

◆ Expression-Based Access Control

Examples:

```
@PreAuthorize("#id == authentication.principal.id")
```

or

```
http.authorizeHttpRequests()
    .requestMatchers("/admin/**").hasAuthority("ROLE_ADMIN")
    .anyRequest().authenticated();
```

PART 6: SECURITY FEATURES

Feature	Description
CSRF Protection	Prevents cross-site request forgery. Enabled by default.
CORS	Manages cross-origin API access.

Feature	Description
Session Fixation Protection	Prevents session ID reuse on login.
Remember-Me	Persistent login cookie for user convenience.
Security Headers	Adds HTTP headers (HSTS, XSS, Cache-Control, etc.).

PART 7: SECURITY CONFIGURATION STYLES

1 Old Style (extends `WebSecurityConfigurerAdapter`) — Deprecated

2 Modern Style (Bean-based DSL) — Preferred since Spring Security 6.

Example:

```
@Configuration
@EnableWebSecurity
public class SecurityConfig {

    @Bean
    SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        http
            .csrf(csrf → csrf.disable())
            .authorizeHttpRequests(auth → auth
                .requestMatchers("/login", "/register").permitAll()
                .anyRequest().authenticated())
            .formLogin(withDefaults());
        return http.build();
    }
}
```

SPRING SECURITY FILTER CHAIN

1. What Is a Filter Chain?

In **Spring Security**, every incoming HTTP request passes through a **chain of security filters** before reaching your controller.

Think of it like an **airport security check** — your request must go through multiple gates (filters) before reaching the destination (controller).

2. Core Idea

Each **Filter** in the chain performs one small security task:

Filter	Responsibility
<code>SecurityContextPersistenceFilter</code>	Restores/Saves SecurityContext for each request
<code>UsernamePasswordAuthenticationFilter</code>	Handles form login authentication
<code>BasicAuthenticationFilter</code>	Handles HTTP Basic authentication
<code>BearerTokenAuthenticationFilter</code>	Handles JWT bearer tokens
<code>ExceptionTranslationFilter</code>	Catches exceptions like AccessDeniedException
<code>FilterSecurityInterceptor</code>	Final authorization check before controller

All these filters are managed by `SecurityFilterChain`, which is a **Spring bean** that defines which filters apply to which URL patterns.

3. The SecurityFilterChain Interface

```
public interface SecurityFilterChain {  
    boolean matches(HttpServletRequest request);  
    List<Filter> getFilters();  
}
```

Meaning:

- **matches()** → checks if this chain applies to a given request (based on URL, etc.).

- **getFilters()** → returns list of filters to apply.

✓ You can have **multiple SecurityFilterChains**, each handling different parts of your app (e.g., one for `/api/**`, another for `/admin/**`).

4. How It Fits Internally

Here's the **flow of an HTTP request** inside Spring Security:

```
Client Request
  ↓
DelegatingFilterProxy (Spring Security entry point)
  ↓
FilterChainProxy
  ↓
SecurityFilterChain
  ↓
→ SecurityContextPersistenceFilter
→ UsernamePasswordAuthenticationFilter
→ BasicAuthenticationFilter
→ CsrfFilter
→ ExceptionTranslationFilter
→ FilterSecurityInterceptor
  ↓
DispatcherServlet (your controller)
```

5. Main Filters in Order (Default Chain Breakdown)

Let's go through the **default filter chain order** in a typical Spring Boot app:

Order	Filter Class	Purpose
1	<code>ChannelProcessingFilter</code>	Ensures HTTPS if required
2	<code>WebAsyncManagerIntegrationFilter</code>	Integrates async web requests with SecurityContext

Order	Filter Class	Purpose
3	<code>SecurityContextPersistenceFilter</code>	Loads & saves <code>SecurityContext</code>
4	<code>HeaderWriterFilter</code>	Adds security headers (X-Frame-Options, etc.)
5	<code>CorsFilter</code>	Handles Cross-Origin Resource Sharing
6	<code>CsrfFilter</code>	Protects against Cross-Site Request Forgery
7	<code>LogoutFilter</code>	Handles /logout endpoint
8	<code>UsernamePasswordAuthenticationFilter</code>	Authenticates form-based login
9	<code>DefaultLoginPageGeneratingFilter</code>	Auto-generates login page (if no custom one)
10	<code>BasicAuthenticationFilter</code>	HTTP Basic auth support
11	<code>RequestCacheAwareFilter</code>	Caches unauthorized requests for redirect after login
12	<code>SecurityContextHolderAwareRequestFilter</code>	Wraps <code>HttpServletRequest</code> with security features
13	<code>AnonymousAuthenticationFilter</code>	Adds anonymous user if no authentication exists
14	<code>SessionManagementFilter</code>	Manages session fixation & concurrency
15	<code>ExceptionTranslationFilter</code>	Converts exceptions to proper responses
16	<code>FilterSecurityInterceptor</code>	Final authorization before reaching controller

6. Key Filters Explained (Internals)

`SecurityContextPersistenceFilter`

- The **first** filter that manages the `SecurityContext`.
- Loads user details from the session and sets it in `SecurityContextHolder`.
- Clears context after request completes.

◆ UsernamePasswordAuthenticationFilter

- Triggered during **form login**.
 - Extracts username and password from request.
 - Delegates to `AuthenticationManager` for verification.
 - If success → stores `Authentication` in `SecurityContext` .
-

◆ BasicAuthenticationFilter

- Looks for `Authorization: Basic <base64>` header.
 - Decodes credentials → authenticates.
 - Sets result into `SecurityContext` .
-

◆ BearerTokenAuthenticationFilter

- For JWT tokens.
 - Looks for `Authorization: Bearer <token>` .
 - Validates and sets authentication.
-

◆ CsrfFilter

- Ensures POST/PUT/DELETE requests include valid CSRF token.
 - Prevents malicious sites from performing unauthorized actions.
-

◆ ExceptionTranslationFilter

- Catches `AuthenticationException` or `AccessDeniedException` .
 - Redirects user to login page or returns 403 response.
-

◆ FilterSecurityInterceptor

- The **last line of defense**.
- Intercepts request and checks if the authenticated user has required authorities for that URL.

- Uses **AccessDecisionManager** to decide allow/deny.

7. Real-Life Flow Example (Form Login)

Let's trace what happens step-by-step when a user logs in:

1. User submits form: `/login` with username & password.
2. `UsernamePasswordAuthenticationFilter` intercepts the request.
3. It builds an `Authentication` token and passes it to `AuthenticationManager`.
4. The manager delegates to `DaoAuthenticationProvider`, which:
 - Calls `UserDetailsService.loadUserByUsername()`
 - Compares passwords using `PasswordEncoder`
5. If successful, user info is stored in `SecurityContextHolder`.
6. `SecurityContextPersistenceFilter` saves it in session.
7. Future requests → same context restored automatically.

8. Custom SecurityFilterChain Configuration

You can define multiple filter chains — for different request patterns.

Example:

```
@Configuration
@EnableWebSecurity
public class SecurityConfig {

    @Bean
    public SecurityFilterChain apiChain(HttpSecurity http) throws Exception {
        http
            .securityMatcher("/api/**")
            .authorizeHttpRequests(auth -> auth
                .anyRequest().authenticated())
            .addFilterBefore(new JwtAuthFilter(), UsernamePasswordAuthenticationFilter.class)
    }
}
```

```

        .csrf(csrf → csrf.disable());
    return http.build();
}

@Bean
public SecurityFilterChain webChain(HttpSecurity http) throws Exception {
    http
        .securityMatcher("/**")
        .authorizeHttpRequests(auth → auth
            .anyRequest().permitAll());
    return http.build();
}
}

```

◆ Here, `/api/**` requests go through JWT filter chain, while web pages use a different security setup.

9. Custom Filter Example

Example custom JWT filter:

```

@Component
public class JwtAuthFilter extends OncePerRequestFilter {
    @Override
    protected void doFilterInternal(HttpServletRequest req, HttpServletResponse res, FilterChain chain)
        throws ServletException, IOException {
        String token = req.getHeader("Authorization");
        if (token != null && token.startsWith("Bearer ")) {
            String jwt = token.substring(7);
            if (validate(jwt)) {
                Authentication auth = getAuthentication(jwt);
                SecurityContextHolder.getContext().setAuthentication(auth);
            }
        }
    }
}

```

```

    }
    chain.doFilter(req, res);
  }
}

```

Then register it:

```

http.addFilterBefore(new JwtAuthFilter(), UsernamePasswordAuthenticationFilter.class);

```

10. Advanced: Multiple Filter Chains (Per URL Segment)

Spring Security supports **multiple independent filter chains**, each with different rules:

```

@Bean
SecurityFilterChain adminChain(HttpSecurity http) throws Exception {
    http
        .securityMatcher("/admin/**")
        .authorizeHttpRequests(auth -> auth.anyRequest().hasRole("ADMIN"))
        .formLogin();
    return http.build();
}

```

```

@Bean
SecurityFilterChain userChain(HttpSecurity http) throws Exception {
    http
        .securityMatcher("/user/**")
        .authorizeHttpRequests(auth -> auth.anyRequest().hasRole("USER"))
        .httpBasic();
    return http.build();
}

```



```
}
```

Requests are matched **in order**, so `/admin/**` uses one chain, `/user/**` another.

11. Debugging the Filter Chain

Add this line in `application.properties` :

```
logging.level.org.springframework.security=TRACE
```

Spring will print out **every filter** applied in the chain on startup, like:

```
Will secure any request with [org.springframework.security.web.FilterChainProxy$VirtualFilterChain@12345]
```

You can also debug dynamically using:

```
SecurityFilterChain chain = http.build();  
chain.getFilters().forEach(f → System.out.println(f.getClass().getName()));
```